



UNIVERSITÀ DI PISA

MSc in Data Science and Business Informatics

BUSINESS PROCESS MODELLING · PROJECT REPORT

P29: Thesis

Francesco Secoli · Student ID 502525 · Academic Year 2025/2026

1	Scenario and approach	1
2	From BPMN modelling to Petri nets	3
2.1	Orchestrations	3
2.2	Collaboration	4
2.3	Deriving the nets	5
3	Analysis	7
3.1	Per pool	8
3.2	The collaboration	8
3.2.1	Free-choice, well-structuredness and S-coverability	8
3.2.2	Safeness and the size of the reachability graph	9
3.2.3	Soundness, weak and relaxed	9
3.2.4	Three modifications, and the limit of the encoding	10

1 Scenario and approach

Consider the (suitably simplified) scenario of the preparation of a degree thesis by a student.

The student contacts a professor, expressing interest in writing a thesis in the professor's area of expertise. The professor proposes a set of possible topics, and the student either picks one or informs the professor that they are no longer interested in the thesis. If the professor has been chosen as supervisor, they send the student some material to study.

After studying the topic, the student starts developing the thesis, periodically exchanging documents, chapter drafts and software prototypes with the supervisor, and scheduling meetings to discuss the work in depth. Each time the student sends an object, the supervisor replies by annotating the object with comments. Each time a meeting must be scheduled, the student proposes a date, which the supervisor may confirm or reject by proposing an alternative date, and so on until a date suits both.

When the thesis is complete, the student sends it to a counter-examiner for evaluation. The counter-examiner may decide to schedule an appointment with the student for a direct presentation of the work, or deem such a meeting unnecessary. The date of the meeting, if any, is set as described above.

The process ends when the supervisor and the counter-examiner send their evaluations to the committee which, after examining the student and reading the evaluations, meets to decide the graduation grade and communicates it to the student.

Design suitable processes that faithfully mirror the scenario described above and that interact correctly.

The assignment above describes the thesis-preparation process with few gaps, and we close each one with a decision of our own.

- **Exchanging objects and scheduling meetings at once** is never addressed, and we keep the two exclusive.
- **No one is appointed to call the thesis complete**, so we give that verdict to the supervisor and leave the student to ask for it.
- **Nothing prompts the two evaluations**, and we fix one for each: the supervisor sends on approving the thesis, the counter-examiner once the presentation is settled or waived.
- **The examination is not ordered against the two evaluations**, and we have the committee convoke the student only after both have arrived.

We model the scenario in BPMN, translate each diagram into a workflow net, and run the same checks twice: on each pool alone ([Section 3.1](#)), then on the four composed ([Section 3.2](#)). The two answers diverge, and that divergence is the report's subject: every pool is sound by itself, while the collaboration is not. [Table 1](#) maps each requirement of the assignment to the place that discharges it, and [Table 2](#) lists the tools.

What the assignment asks	Where
the design choices	Sections 1 and 2
processes that faithfully mirror the scenario	Section 2
processes that interact correctly	Sections 2.2 and 3.2
models drawn at an abstract level, then transformed into workflow nets	Sections 2 and 2.3
how the transformation was obtained	Section 2.3
invariants, S-components, free-choice, well-handled (well-structuredness), safeness	Sections 3.1 and 3.2
an estimate of the reachability-graph size in vertices and arcs	Sections 3.1 and 3.2
soundness of each participant's orchestration	Section 3.1
soundness of the whole collaboration	Section 3.2
if no sound overall process is possible: the reason, with relaxed and weak soundness	Section 3.2
diagrams and nets in a standard interchange format	bpmn/, pnml/
analysis output and reachability-graph images	pnml/analysis/, pnml/img/

Table 1: Every requirement of the assignment against the place that discharges it.

Step and tool
1. Drawing (bpmn.io) Visual Studio Code extension, editing native BPMN 2.0 XML.
2. Validation (bpmnlint 11.12.1) bpmnlint:all in <code>scripts/lint/.bpmnlintrc</code> , run by <code>npm --prefix scripts/lint run lint</code> .
3. Transformation (bpmn2petrinet) Browser tool, cloned from BenjaNapo/bpmn-to-petri and driven headless by <code>scripts/03_to_pnml.py</code> with its <code>withCollapsedXor</code> setting on.
4. Analysis (pm4py 2.7.23.6) pm4py 's reimplementaion of Woflan reached as a library by <code>scripts/04_pm4py_report.py</code> . WoPeD spreads the same figures across panels, so collecting them would take a script reading numbers off a screen.
5. Rendering (pm4py and WoPeD) <code>scripts/04_pm4py_report.py</code> draws each net and its reachability graph, WoPeD the same nets in a second layout, all under <code>pnml/img/</code> .

Table 2: The pipeline from diagram to analysis.

2 From BPMN modelling to Petri nets

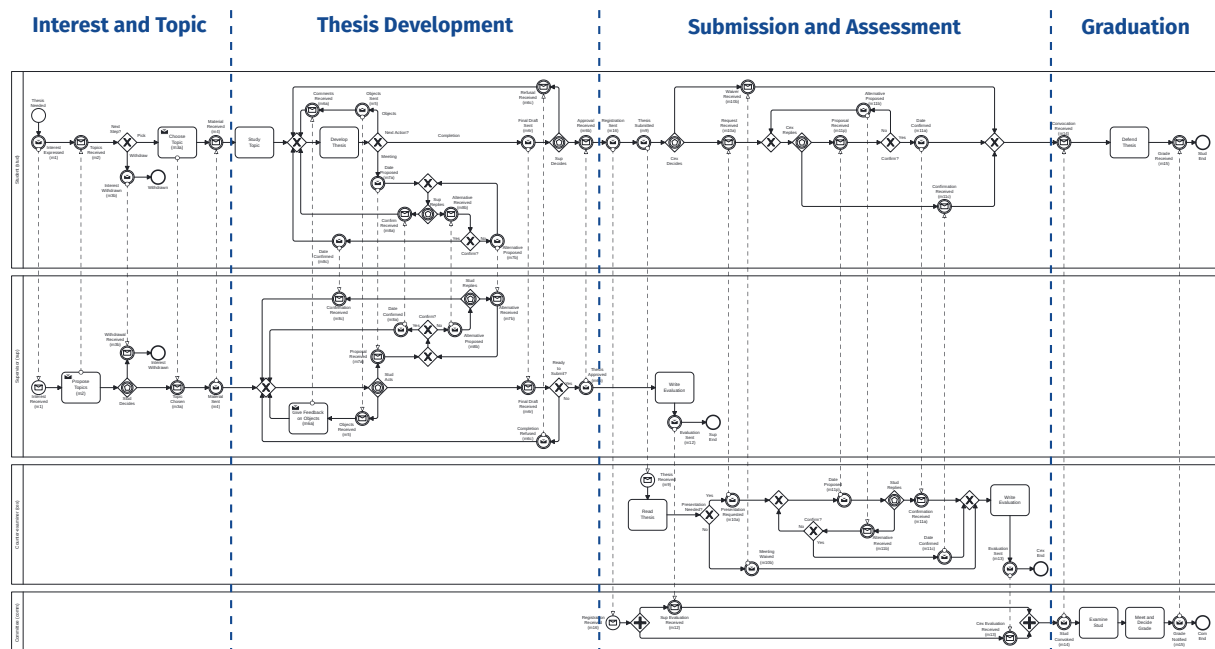


Figure 1: The collaboration (bpmn/collaboration.bpmn), cut into the four phases of [Section 2.2](#).

Figure 1 is the whole model: one row per participant, four orchestrations in all ([Section 2.1](#)), running together through the four phases of the collaboration ([Section 2.2](#)). No wait in it carries a deadline, since we assume every participant answers in the end, so the model has no timer and no branch for an answer that never comes.

2.1 Orchestrations

In each orchestration, and in the collaboration, a joint act takes a task only where the messages alone do not show it happened. When both participants send or receive the message that settles it, as with the acceptance of a meeting date, it takes none. If only one side sends, as with the convocation to the examination, nothing shows that the student attended, and the act takes a task on each side. [Table 3](#) counts the elements of each orchestration.

	Student	Supervisor	Counter-examiner	Committee	Total
start / end events	1 / 2	1 / 2	1 / 1	1 / 1	4 / 6
tasks (send tasks)	4 (1)	3 (2)	2 (0)	2 (0)	11 (3)
intermediate catch events	13	7	2	2	24
intermediate throw events	11	6	5	2	24
event-based gateways	4	3	1	0	8
exclusive gateways	8	4	4	0	16
parallel gateways	0	0	0	2	2
nodes	43	26	16	10	95
sequence flows	50	30	18	10	108

Table 3: Element counts for the four orchestrations.

Student (`bpmn/student.bpmn`). Each round of the development loop opens on Develop Thesis and reaches the XOR-split Next Action?, which offers three moves towards the supervisor: send an object, propose a meeting date, or send the final draft and ask whether the work is done. Keeping them exclusive rules out the runs where the supervisor is annotating a draft while a date is still open, and it leaves the pool an S-net.

Once the supervisor approves, the student registers for the graduation call and only then sends the thesis to the counter-examiner. The student waits to hear whether the counter-examiner wants a presentation, confirms or counters the date they propose, and ends on the convocation, Defend Thesis and the grade.

Supervisor (`bpmn/supervisor.bpmn`). The student's withdrawal closes this orchestration at once. Otherwise it mirrors the student's, with one wait for the three branches a round can take and another after the supervisor counter-proposes a date. It ends when the supervisor writes the evaluation and sends it to the committee.

Counter-examiner (`bpmn/counter-examiner.bpmn`). Read Thesis precedes the XOR-split Presentation Needed?. The counter-examiner either waives the meeting or asks for one and proposes a date, which the student confirms or counters until both agree. The branches merge, and Write Evaluation follows, so whatever the presentation settles reaches the evaluation.

Committee (`bpmn/committee.bpmn`). Nothing orders the supervisor's evaluation against the counter-examiner's, and the assignment makes the examination and the reading two prerequisites of the same meeting, so the orchestration waits for the two evaluations in parallel and joins before convoking the student. That is the one AND in the model.

2.2 Collaboration

Twenty-seven message flows wire the four pools in the collaboration ([Table 4](#)). The student's pool is the axis: it touches twenty-five, missing only the two evaluations that reach the committee. The student also instantiates all three of the others, one message each. Because that pool runs through all of it, the collaboration reads as four phases, drawn over [Figure 1](#) as bands rather than as part of the diagram.

Interest and Topic. The student contacts the supervisor, who replies with a list of topics and from then on only reacts, since nothing in that orchestration starts on its own. The student picks a topic or withdraws, the only point at which the case can be abandoned.

Thesis Development. The core of the model is a loop of request and response. The student and the supervisor step together, one message and its answer per round, and the loop ends on the supervisor's verdict, so both leave development together.

Submission and Assessment. The registration instantiates the committee, and the thesis the counter-examiner. The supervisor's evaluation could have started the committee instead; the registration keeps the student the only one who starts another participant.

Graduation. The committee convokes the student and the two hold the examination, a task in each pool. The committee then decides the grade and sends it, closing its own orchestration and the student's.

#	Msg	From	To	Content	Phase	In the assignment
1	<i>m₁</i>	stud	sup	expression of interest	Interest and Topic	✓
2	<i>m₂</i>	sup	stud	topic list	Interest and Topic	✓
3	<i>m_{3a}</i>	stud	sup	topic choice	Interest and Topic	✓
4	<i>m_{3b}</i>	stud	sup	withdrawal	Interest and Topic	✓
5	<i>m₄</i>	sup	stud	study material	Interest and Topic	✓
6	<i>m₅</i>	stud	sup	draft / object	Thesis Development	✓
7	<i>m_{6a}</i>	sup	stud	comments, keep going	Thesis Development	✓
8	<i>m_{6b}</i>	sup	stud	ready to submit	Thesis Development	
9	<i>m_{6c}</i>	sup	stud	not yet, keep going	Thesis Development	
10	<i>m_{6r}</i>	stud	sup	final draft and completion request	Thesis Development	
11	<i>m_{7a}</i>	stud	sup	meeting-date proposal	Thesis Development	✓
12	<i>m_{7b}</i>	stud	sup	alternative date	Thesis Development	✓
13	<i>m_{8a}</i>	sup	stud	date confirmed	Thesis Development	✓
14	<i>m_{8b}</i>	sup	stud	counter-proposal	Thesis Development	✓
15	<i>m_{8c}</i>	stud	sup	counter-date accepted	Thesis Development	
16	<i>m₉</i>	stud	cex	completed thesis	Submission and Assessment	✓
17	<i>m_{10a}</i>	cex	stud	presentation requested	Submission and Assessment	✓
18	<i>m_{10b}</i>	cex	stud	no meeting needed	Submission and Assessment	
19	<i>m_{11p}</i>	cex	stud	presentation-date proposal	Submission and Assessment	✓
20	<i>m_{11a}</i>	stud	cex	date confirmed	Submission and Assessment	✓
21	<i>m_{11b}</i>	stud	cex	counter-proposal	Submission and Assessment	✓
22	<i>m_{11c}</i>	cex	stud	counter-date accepted	Submission and Assessment	
23	<i>m₁₂</i>	sup	comm	supervisor evaluation	Submission and Assessment	✓
24	<i>m₁₃</i>	cex	comm	counter-examiner evaluation	Submission and Assessment	✓
25	<i>m₁₆</i>	stud	comm	graduation registration	Submission and Assessment	
26	<i>m₁₄</i>	comm	stud	exam convocation	Graduation	
27	<i>m₁₅</i>	comm	stud	grade	Graduation	✓

Table 4: Message inventory: twenty-seven flows in the delivered model, each with exactly one sender and one receiver, so the four modules are structurally compatible and compose. **stud** is the student, **sup** the supervisor, **cex** the counter-examiner and **comm** the committee. The last column marks whether the assignment names a message; the unmarked ones are ours.

2.3 Deriving the nets

`bpmnlint` reports nothing on the six diagrams, and `bpmn2petrinet` then translates them with `withCollapsedXor` on. Every exclusive and event-based gateway collapses into one place, and its branches compete on that place; every other node becomes a transition. A sequence flow becomes a place between two transitions, or a plain arc where a collapsed gateway sits at one end. We drive the converter headless from `scripts/03_to_pnml.py`, so no step between diagram and net is manual.¹

The tool builds the workflow net itself. It puts a source place in front of the start event and marks that place with one token, and it merges the end events of a process into a single sink, so the six pool ends become four. Every message flow gets one place, shared by the sending and the receiving transition and reused on every round. One transition then closes the collaboration, collecting the four pool sinks into a final place.

`scripts/01_extract_pools.py` cuts each participant out of `bpmn/collaboration.bpmn` into a diagram of its own, so the analysis has both the collaboration and the four orchestrations from the same model. Read alone, an orchestration carries a source of its own and no message places; inside the collaboration the message places return, and the student's is the only source.

¹The converter departs from the PNML schema in two places: it writes `<position>` where the schema asks for `<offset>`, and it leaves the `<net>` without the `id` and `type` the standard requires. We repair both. Neither changes the analysis, since `pm4py` reads the file either way; they are there for the tools that validate before reading and return nothing on a file that fails.

Table 5 reconciles the two readings, and its transitions follow from **Table 3** up to one correction: a flow that runs from one collapsed gateway into another would leave two places adjacent, so it becomes a transition of its own. There are two such flows in the student, two in the supervisor, one in the counter-examiner and none in the committee, which gives $43 - 12 + 2 = 33$ from 43 nodes and 12 collapsed gateways, and the same arithmetic for the rest. The places follow the same way: one for each collapsed gateway, one for each flow left with a transition at both ends, and the source and the sink, so $12 + 11 + 2 = 25$ for the student.

	Source	Places	Transitions	Arcs
Student	pnml/student.pnml	25	33	66
Supervisor	pnml/supervisor.pnml	16	21	42
Counter-examiner	pnml/counter-examiner.pnml	10	12	24
Committee	pnml/committee.pnml	12	10	22
the four pools		63	76	154
sources the message-started pools lose		-3	0	-3
the twenty-seven message places		+27	0	+54
closure, the final place and its transition		+1	+1	+5
collaboration	pnml/collaboration.pnml	88	77	210

Table 5: The final nets. The three middle rows reconcile the system with the four pools.

3 Analysis

Six nets face the same list of checks (**Table 6**): the four pools, the delivered collaboration, and one variant of it. `scripts/04_pm4py_report.py` runs them through `pm4py` and writes one JSON report each into `pnml/analysis/`.²

	<i>per pool</i>				<i>collaboration</i>	
	Student	Superv.	C-ex.	Comm.	delivered	variant
<i>Size</i>						
places / transitions / arcs	25/33/66	16/21/42	10/12/24	12/10/22	88/77/210	85/73/200
reachability graph of N^*	25/34	16/22	10/13	12/13	712/1983	706/1975
<i>Workflow-net shape, on N</i>						
workflow net	✓	✓	✓	✓	✓	✓
properly initialised	✓	✓	✓	✓	✓	✓
<i>Structural class</i>						
S-net, on N	✓	✓	✓	✗	✗	✗
T-net, on N^*	✗	✗	✗	✓	✗	✗
free-choice	✓	✓	✓	✓	✗	✗
<i>Structure of N^*</i>						
PT- / TP-handles	0/0	0/0	0/0	0/0	214/117	178/74
well-structured	✓	✓	✓	✓	✗	✗
S-coverable	✓	✓	✓	✓	✗	✓
positive S-invariant	✓	✓	✓	✓	✗	✓
positive T-invariant	✓	✓	✓	✓	✗	✓
Rank Theorem	✓	✓	✓	✓	—	—
rank / clusters	24/25	15/16	9/10	10/11	69/60	65/58
<i>Verdict</i>						
safe	✓	✓	✓	✓	✓	✓
no dead tasks	✓	✓	✓	✓	✓	✓
live and bounded, on N^*	✓	✓	✓	✓	✗	✓
option to complete	✓	✓	✓	✓	✗	✓
proper completion	✓	✓	✓	✓	✓	✓
relaxed sound	✓	✓	✓	✓	✗	✓
sound	✓	✓	✓	✓	✗	✓

Table 6: Every net the report analyses. *Variant* deletes the withdrawal branch and changes nothing else; it was built to test the diagnosis, not proposed (**Section 3.2.4**). N^* is the short-circuit of the net in its own column. The Rank Theorem row is its four non-trivial conditions, since the other two hold for any connected net with a place and a transition, and it is blank where its free-choice hypothesis fails. The pipeline reports every row but the three above *sound*, which follow from the Rank Theorem for the pools and from the reachability graph for the collaborations. Only the T-invariant row carries a verdict, and in one direction: on a bounded net its absence rules soundness out, its presence proves nothing.

²Five of the results in those reports are the script's own and not Woflan's: the S-components, the Rank Theorem conditions, the free-choice verdict with its list of violations, the S-net and T-net verdicts, and the split of the not-well-handled pairs into PT and TP.

3.1 Per pool

The student, the supervisor and the counter-examiner branch on places alone, at their exclusive gateways and their event-based waits alike: each is an S-net, and so safe and sound. Each N^* is a strongly connected S-net, so its whole place set forms a single S-component, and that component's uniform invariant, all ones, is positive. One token then marks one place at a time, so the reachability graph takes a vertex per place and an arc per transition of N^* , reset included. The 25/34, 16/22 and 10/13 of [Table 6](#) follow from the place and transition counts in the same table.

The committee branches on transitions instead, at its one AND split and join, so its N^* is a T-net. Its AND split marks two places at once, so the arc half of that correspondence fails: thirteen arcs against eleven transitions. The vertex half survives, but by arithmetic alone. Eight singleton markings run along the shared spine, and the two states of each evaluation arm combine into four more: twelve in all, as many as the places.

Neither shape admits a handle. A handle of either kind needs a branching place at one end and a branching transition at the other. An S-net has no branching transition, a T-net no branching place, so the pools carry 0/0 and are well-structured ([Table 6](#)). All four are free-choice, so the Rank Theorem applies and decides liveness and boundedness of N^* in polynomial time. Its four non-trivial conditions hold on each: no proper siphon starts unmarked, a positive S-invariant and a positive T-invariant exist, and the rank of the incidence matrix is one below the cluster count.

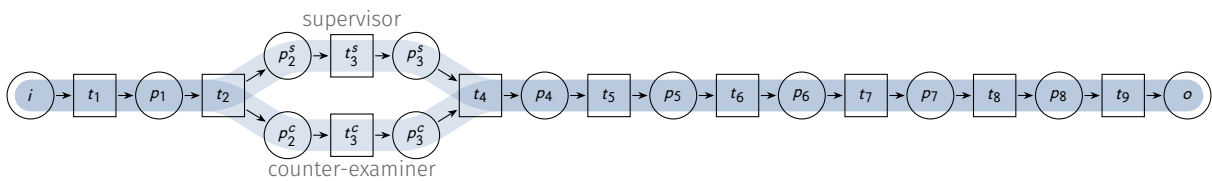


Figure 2: The committee's workflow net, with the names this section uses. The two tinted tracks are its two S-components, one per arm. Its reachability graph runs two edges past the transitions of N^* , because each evaluation receipt fires in either order.

Two S-components cover the committee ([Figure 2](#)), a shared spine and one evaluation arm each. Each has the uniform invariant of its own ten places, weight 1 there and 0 on the other arm. The two sum to 2 along the spine and 1 on each arm, the positive S-invariant the Rank Theorem asks for. The net is acyclic, so every circuit of N^* closes through the reset transition and the marked source. Marked circuits are the one hypothesis the T-net theorem needs, and it settles safe and sound by the dual route.

3.2 The collaboration

Composed into one collaboration, the four pools keep none of the three structural properties they have alone, and the system is not sound ([Table 6](#)).

3.2.1 Free-choice, well-structuredness and S-coverability

Free-choice fails at the eight event-based waits. Alone, the receives of a wait share only that gateway's place, so their presets are equal. Composition gives every send a second output and

every receive a second input, and the presets then overlap without being equal at ten pairs of receives.

Well-structuredness fails at message passing. Every TP-handle starts at a send. Most PT-handles end on a receive, the rest at the closure and at the committee's join. One of those runs from the student's withdrawal choice to the closure, an exclusive split synchronised by an AND-join. That shape is the **deadlock**: the split delivers on one path, the join waits for both ([Section 3.2.3](#)).

S-coverability fails at four places of N^* : the counter-examiner's and the committee's sinks, and the place before each. They lie in the support of no semi-positive S-invariant, and so in no S-component. Woflan reads its S-components off an invariant basis and overcounts. It names ten places here where four lie outside every S-component, and two on the variant where none do. Those four come from putting the question place by place. The missing S-cover shows nothing on its own: both theorems that would read a verdict from it need free-choice or well-structuredness, and neither holds.

3.2.2 Safeness and the size of the reachability graph

Safeness survives, and with it boundedness. Each pool is safe alone ([Table 6](#)). Composition adds the 27 message places and the final place, which only the closure marks. Sixteen of the message places lie on no circuit, and neither do their producers, so those sends fire once per case. The other eleven are kept safe by the strict alternation of one send and one reply on the development channels and by the propose-and-wait shape of the date negotiation itself ([Section 2](#)). That exhausts what composition adds, and none of those places reaches two tokens.

A marking of the composition is one marking per pool plus the messages in flight. Each pool's share of it, when not empty, is a marking that pool reaches alone. Three pools have no source place inside the collaboration ([Table 5](#)), so their empty state takes the slot their standalone initial marking held and their counts do not move. The student keeps its source and gains the empty marking the closure leaves behind. The four graph sizes of [Table 6](#) then bound the shares: at most $26 \cdot 16 \cdot 10 \cdot 12 = 5.0 \cdot 10^4$ combinations, a bound on those and not on $|V|$. On N^* the graph has $|V| = 712$, so at most one combination in seventy is ever reached, and $|E| = 1983$ gives 2.8 arcs per marking. Synchronisation is what collapses the count: a pool advances only when its partner has moved. The pools do not take turns, since in most markings all four hold a token and still cannot move independently.

3.2.3 Soundness, weak and relaxed

N^* is safe, and so bounded: no place holds two tokens in any of its 712 markings. It has no positive T-invariant ([Table 6](#)), and a live and bounded system has one, so N^* is not live. A workflow net is sound exactly when its short circuit is live and bounded: the collaboration is not sound.

Four transitions make the withdrawal branch. The student withdraws and closes, the supervisor receives it and closes, and their two sinks are the only marked places left. The thesis instantiates the counter-examiner, and the graduation registration instantiates the committee. A withdrawing student sends neither, so neither pool ever starts. The closure asks for all four sinks and finds two, so nothing can fire and the final place stays unreachable. No other marking of N^* is dead.

That failure is *option to complete*, the clause asking every reachable marking to leave a path to

the final place. The other two hold: no task is dead (**Table 6**), and *proper completion* survives because the single reachable marking that marks it carries no other token. *Weak soundness* forgives dead tasks and nothing else, a licence this net never needed, so it fails on the same clause. *Relaxed soundness* drops all three and asks each transition for one run from the source to the final place; the four transitions of the withdrawal branch have none, by the run just given.

3.2.4 Three modifications, and the limit of the encoding

The closure is not the cause. A workflow system closes on both ends, and its source transition would mark every pool's initial place, so the counter-examiner and the committee would hold a token on every run. On the withdrawal branch that token waits at their message start event and never moves, and the same deadlock stands. The join is forced too: all six pool end events are plain, so nothing else can mark the final place. A net without it has four sinks and is not a workflow net at all. Three modifications would remove the deadlock. Two add machinery the assignment does not describe; the third removes a case it does describe, and only that one exists as a net, `bpmn/collaboration-no-withdrawal.bpmn`.

- **Loop the withdrawal back to the topic proposal** instead of ending there. The branch no longer reaches a sink, so nothing waits on it. But the assignment has the student lose interest in the thesis, not in the list. Looping sends them back to the same supervisor for another topic, and an exit becomes a second round the text never describes.
- **Add two messages** telling the counter-examiner and the committee that the case is closed. Each pool is then instantiated only to end at once, so all four sinks are marked and the closure fires. But on that branch the student has engaged neither, so the closing message would be their first contact.
- **Delete the withdrawal branch.** The net is then sound (**Table 6**): the S-cover and both invariants return, while free-choice and well-structuredness stay broken. Message passing is what breaks them, and the withdrawal accounts for one of the ten pairs. It invents no round and no channel, and the case it removes sits ahead of the core, before a topic is even picked. But the assignment states that case in the same sentence as the one it keeps, so the net is sound but not faithful to it.

No model that leaves two of the four sinks unmarked and still joins all four can be sound. A model that keeps every case the assignment states, inventing no message and no round, leaves two sinks unmarked and joins all four. The withdrawal is in the assignment, no message it describes starts the counter-examiner or the committee on that branch, and the transformation closes a collaboration by joining every pool sink into one final place. That join is also a limit on what the net can say. Soundness asks for one final marking, and joining the four sinks is the only way this collaboration can report completion. An unmarked sink then reads the same whether its pool never started or has not finished. The withdrawn case ends cleanly for the two participants it involves, and the encoding cannot say so.